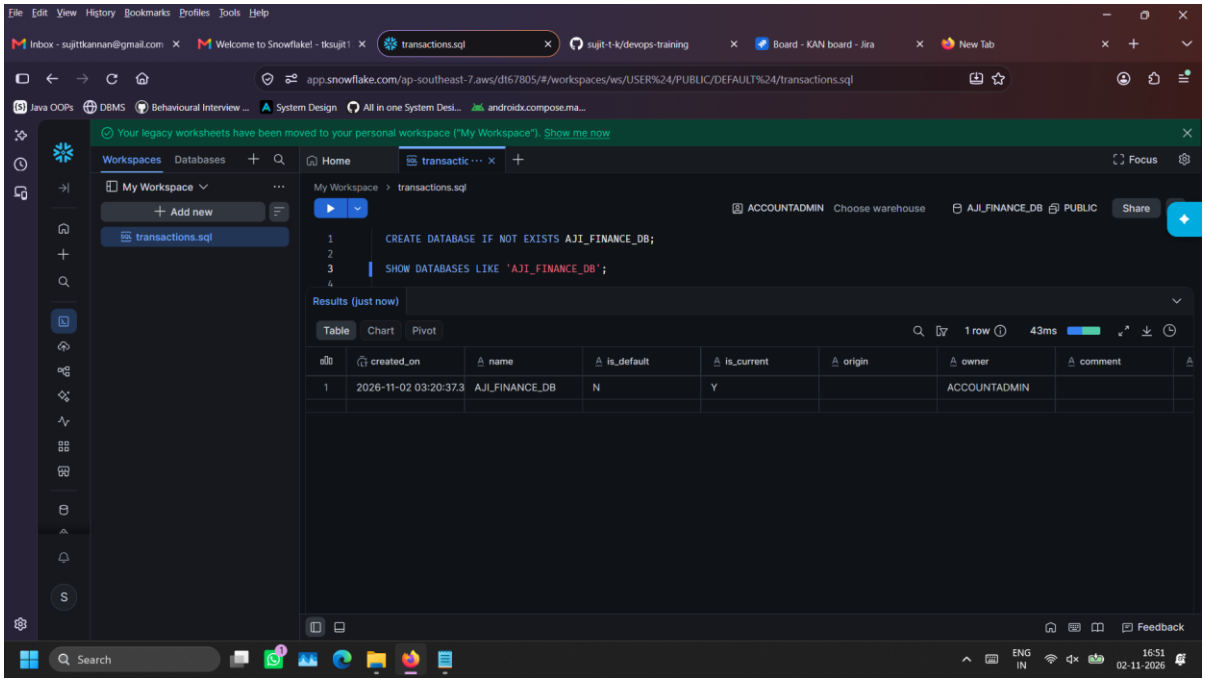
Day: 15 (02-11-2026)	**Snowflake – Load csv file content into Snowflake table**	Documented By:
Task – 2		Sujit T K

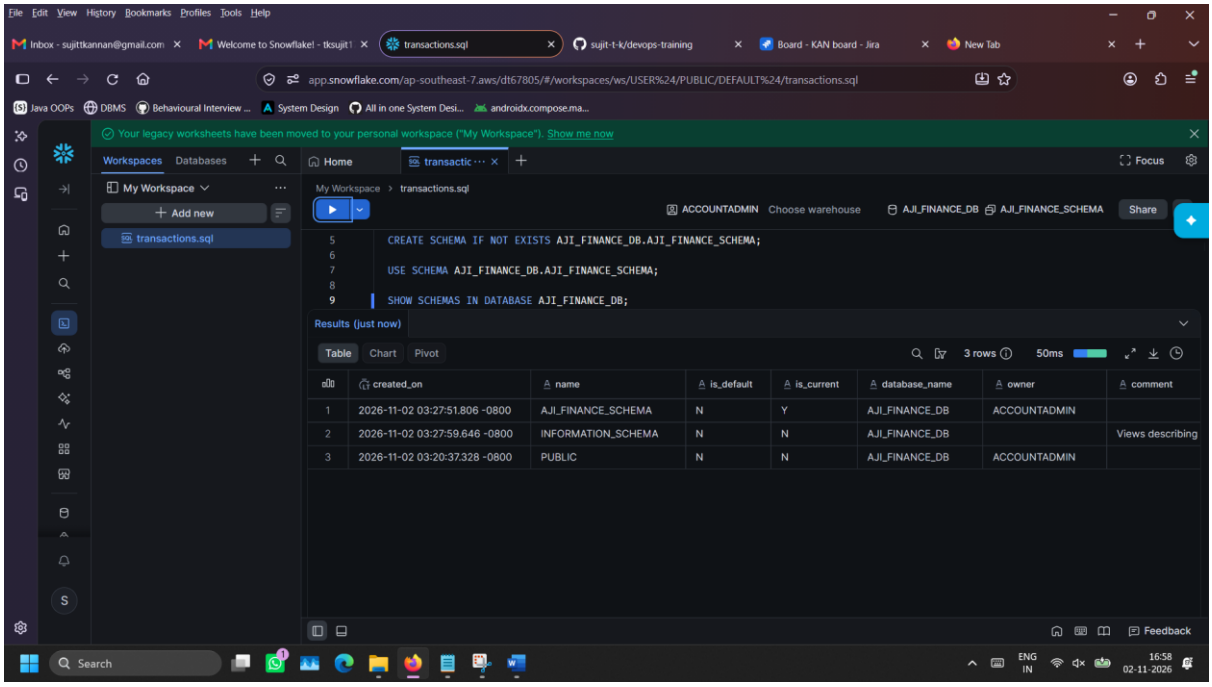
1.) Create a database (to hold data objects):

```
CREATE DATABASE IF NOT EXISTS AJI_FINANCE_DB;
SHOW DATABASES LIKE 'AJI_FINANCE_DB'; //to verify the creation of database
```



2.) Create schema (container within database that can hold tables):

```
CREATE SCHEMA IF NOT EXISTS AJI_FINANCE_DB.AJI_FINANCE_SCHEMA;
USE SCHEMA AJI_FINANCE_DB.AJI_FINANCE_SCHEMA; //to select current schema
SHOW SCHEMAS IN DATABASE AJI_FINANCE_DB; //to verify creation of schema
```

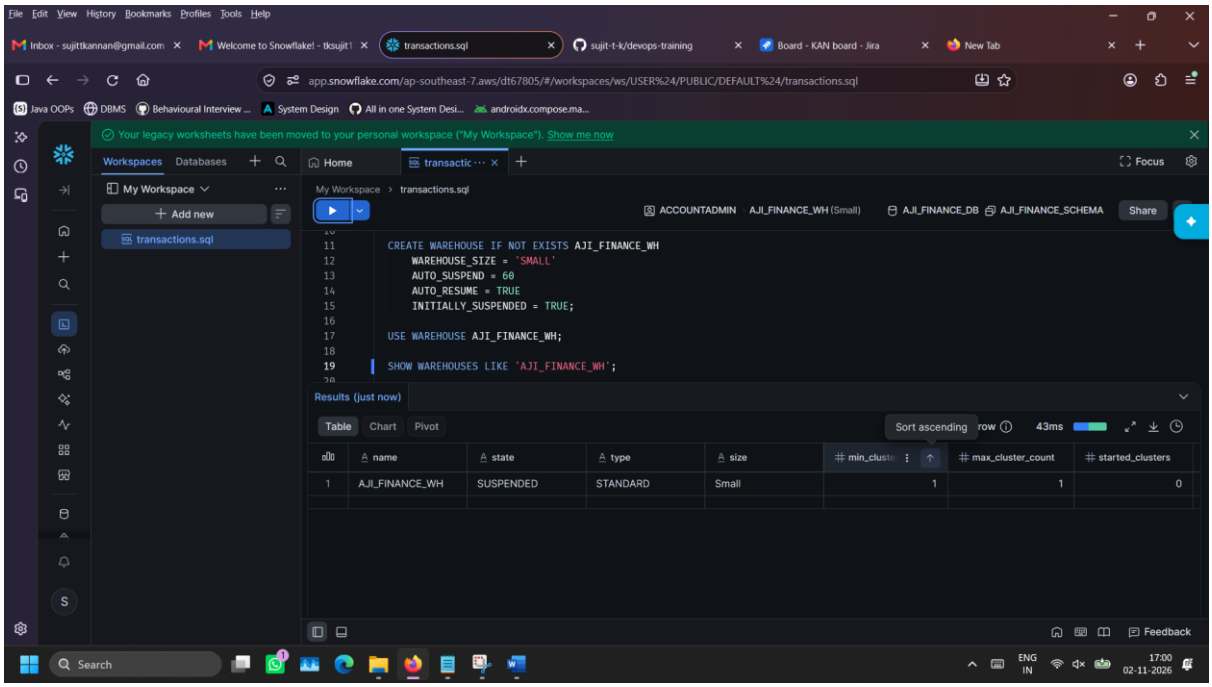


3.) Warehouse creation (to store data):

```
CREATE WAREHOUSE IF NOT EXISTS AJI_FINANCE_WH
    WAREHOUSE_SIZE = 'SMALL'
    AUTO_SUSPEND = 60
    AUTO_RESUME = TRUE
    INITIALLY_SUSPENDED = TRUE;

USE WAREHOUSE AJI_FINANCE_WH; //selects as current warehouse

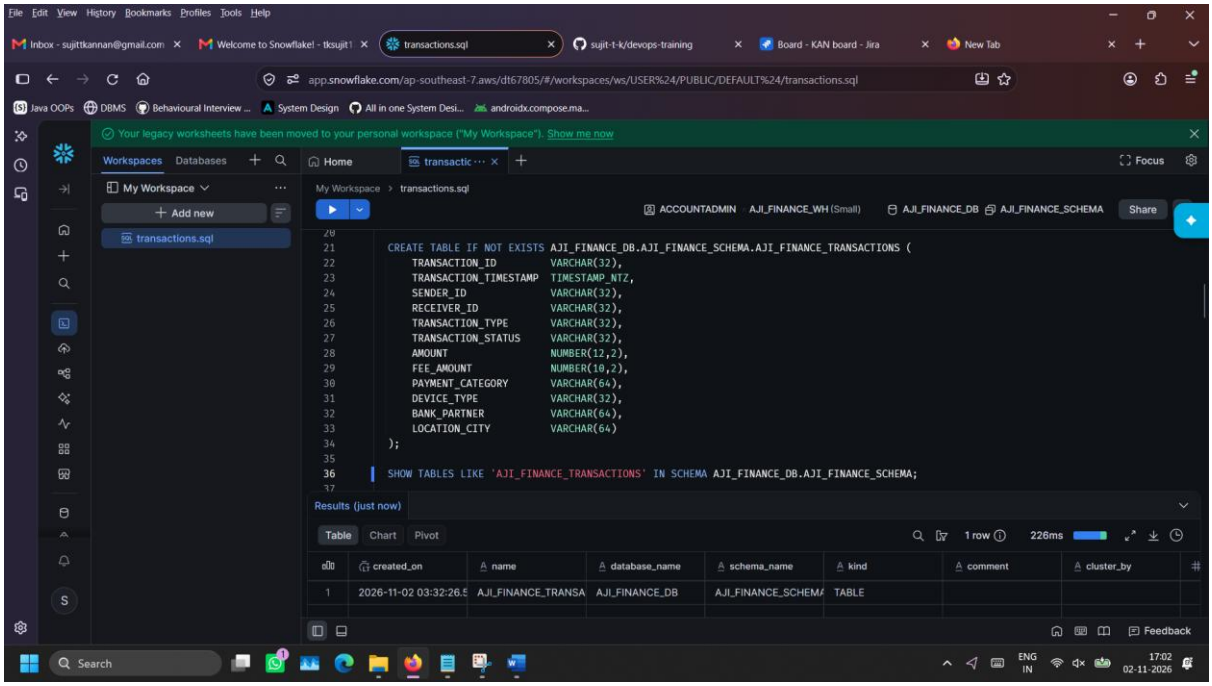
SHOW WAREHOUSES LIKE 'AJI_FINANCE_WH'; //to verify creation of warehouse
```



4.) Creating table in Snowflake: (to orgnaise and store records):

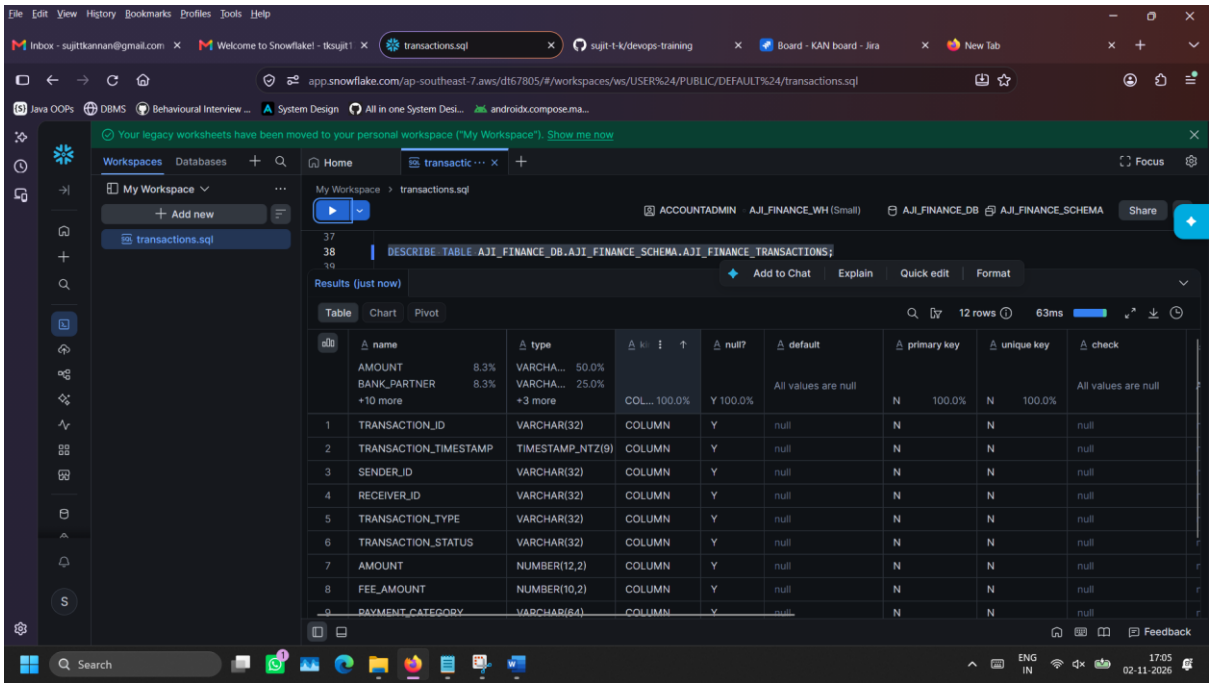
```
CREATE TABLE IF NOT EXISTS
AJI_FINANCE_DB.AJI_FINANCE_SCHEMA.AJI_FINANCE_TRANSACTIONS (
    TRANSACTION_ID          VARCHAR(32),
    TRANSACTION_TIMESTAMP   TIMESTAMP_NTZ,
    SENDER_ID               VARCHAR(32),
    RECEIVER_ID             VARCHAR(32),
    TRANSACTION_TYPE        VARCHAR(32),
    TRANSACTION_STATUS      VARCHAR(32),
    AMOUNT                  NUMBER(12,2),
    FEE_AMOUNT              NUMBER(10,2),
    PAYMENT_CATEGORY        VARCHAR(64),
    DEVICE_TYPE             VARCHAR(32),
    BANK_PARTNER            VARCHAR(64),
    LOCATION_CITY           VARCHAR(64)
);
```

```
SHOW TABLES LIKE 'AJI_FINANCE_TRANSACTIONS' IN SCHEMA
AJI_FINANCE_DB.AJI_FINANCE_SCHEMA; //to verify creation of table
```



5.) Inspecting the table (to view the column names and datatype)

```
DESCRIBE TABLE AJI_FINANCE_DB.AJI_FINANCE_SCHEMA.AJI_FINANCE_TRANSACTIONS;
```

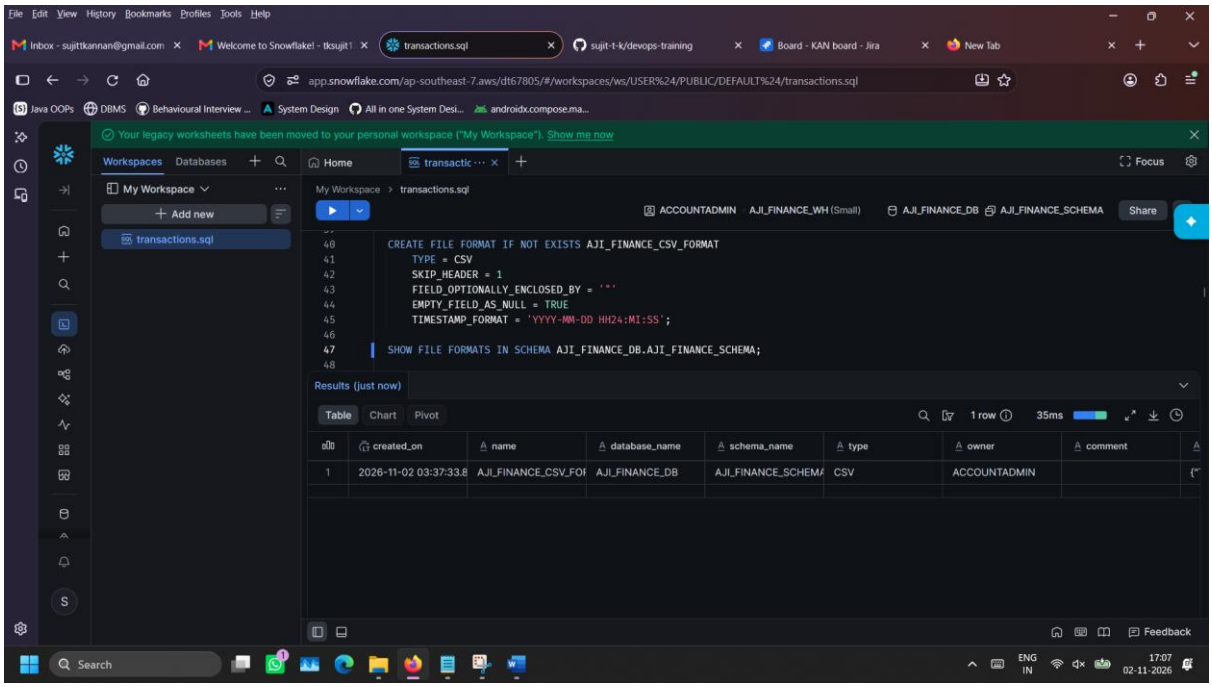


6.) File format creation: (Mention the file format that can be imported into the snowflake stage)

```
CREATE FILE FORMAT IF NOT EXISTS AJI_FINANCE_CSV_FORMAT
    TYPE = CSV
    SKIP_HEADER = 1
    FIELD_OPTIONALLY_ENCLOSED_BY = '"'
```

```
    EMPTY_FIELD_AS_NULL = TRUE
    TIMESTAMP_FORMAT = 'YYYY-MM-DD HH24:MI:SS';
```

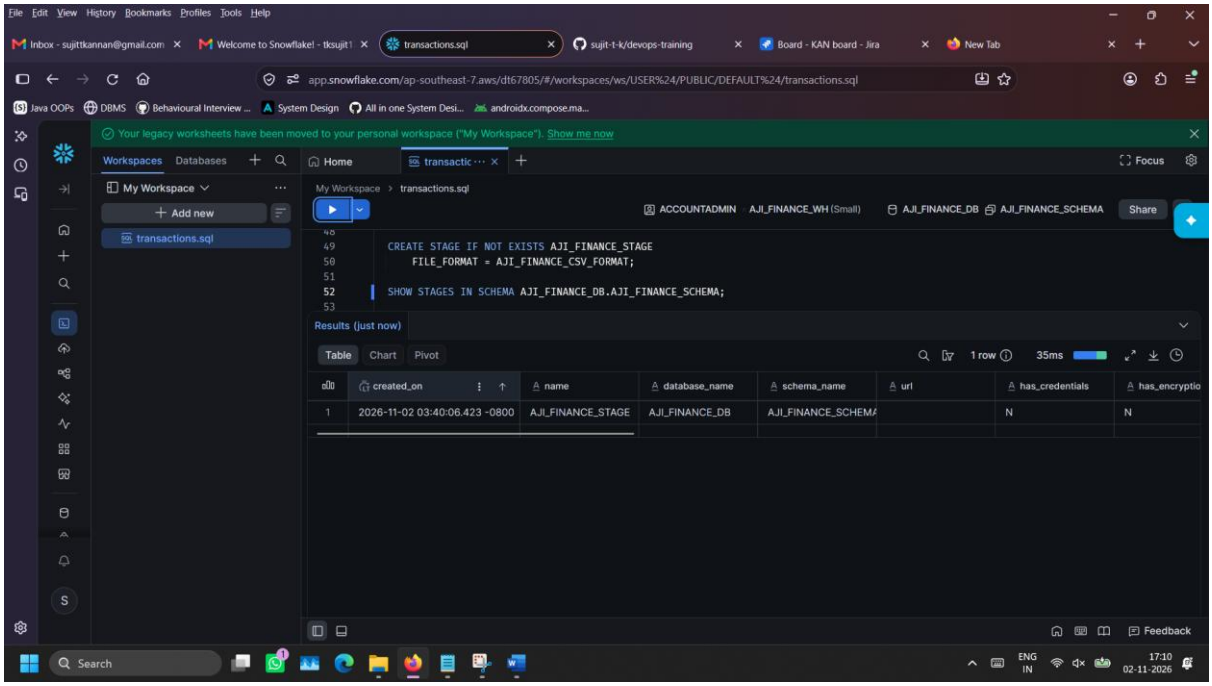
```
SHOW FILE FORMATS IN SCHEMA AJI_FINANCE_DB.AJI_FINANCE_SCHEMA; //to ensure the creation of the file format
```



7.) Creating stage (a location where external file can be uploaded into snowflake environment):

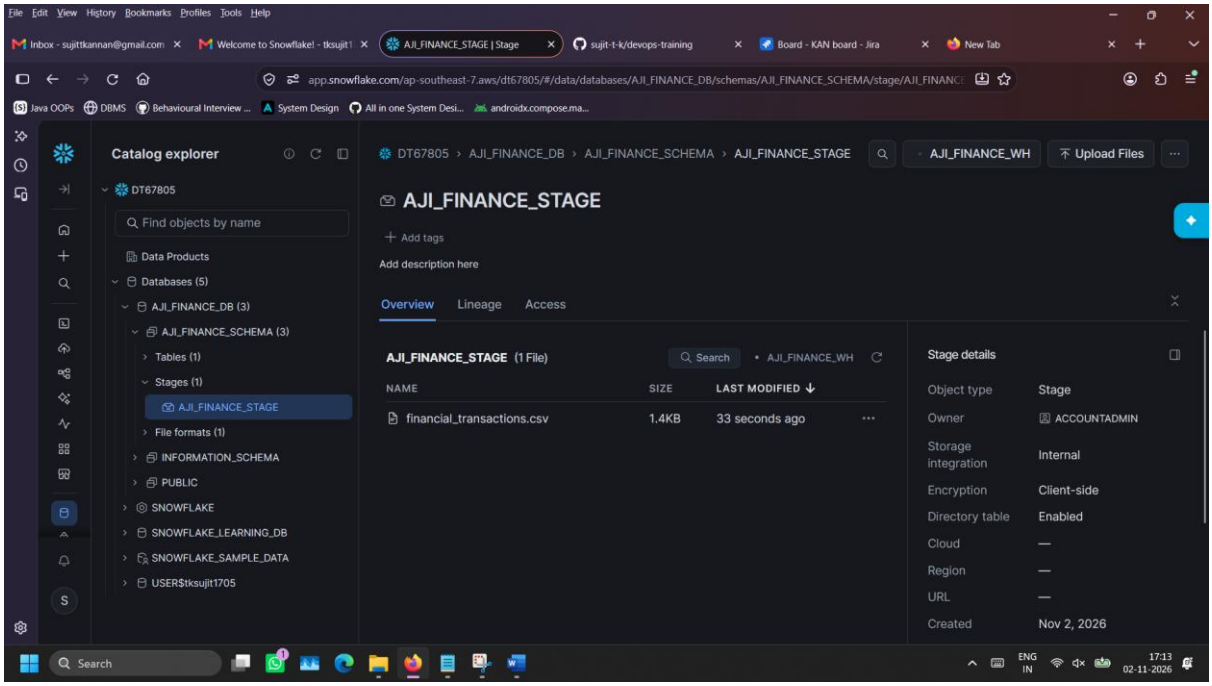
```
CREATE STAGE IF NOT EXISTS AJI_FINANCE_STAGE
    FILE_FORMAT = AJI_FINANCE_CSV_FORMAT;
```

```
SHOW STAGES IN SCHEMA AJI_FINANCE_DB.AJI_FINANCE_SCHEMA;//to verify the creation of the stage
```



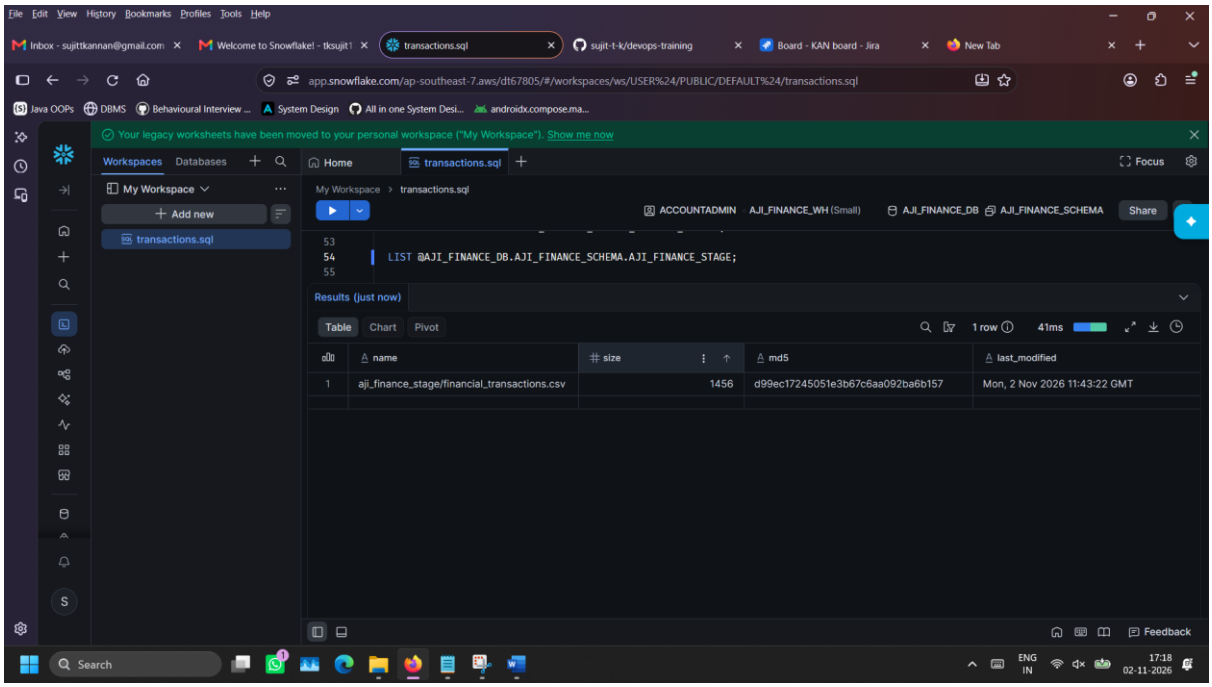
8.) Upload .csv file into AJI_FINANCE_STAGE (so that the .csv file can be accessed internally later):

On left side bar, go to Catalog -> Explorer -> Databases -> AJI_FINANCE_DB -> AJI_FINANCE_SCHEMA -> Stages. Click on AJI_FINANCE_STAGE. Then on top-right “Upload Files” button would be there. Click on it and locate the csv file. After successful upload, the csv file would be listed in the stage.



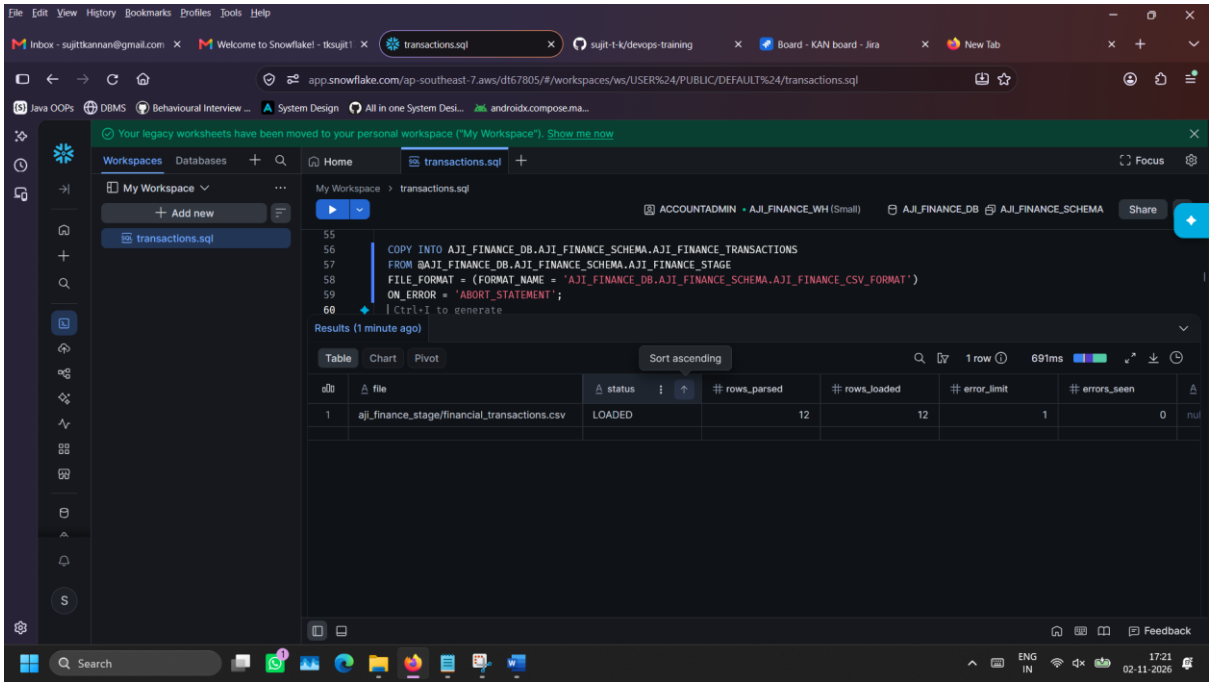
9.) Verification of file uploaded into the stage: (verify using SQL)

```
LIST @AJI_FINANCE_DB.AJI_FINANCE_SCHEMA.AJI_FINANCE_STAGE;
```



10.) Transfer content from stage's csv file to AJI_FINANCE_TRANSACTIONS table:

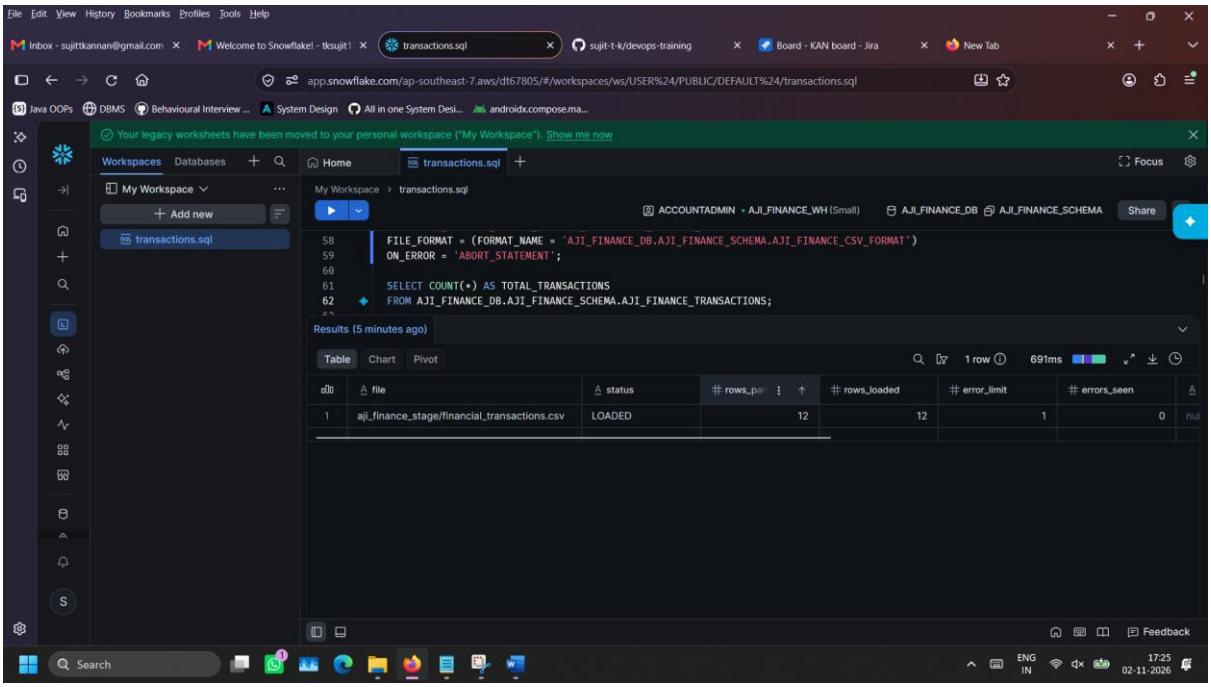
```
COPY INTO AJI_FINANCE_DB.AJI_FINANCE_SCHEMA.AJI_FINANCE_TRANSACTIONS
FROM @AJI_FINANCE_DB.AJI_FINANCE_SCHEMA.AJI_FINANCE_STAGE
FILE_FORMAT = (FORMAT_NAME = 'AJI_FINANCE_DB.AJI_FINANCE_SCHEMA.AJI_FINANCE_CSV_FORMAT')
ON_ERROR = 'ABORT_STATEMENT';
```



11.)Execute SQL queries on the table:

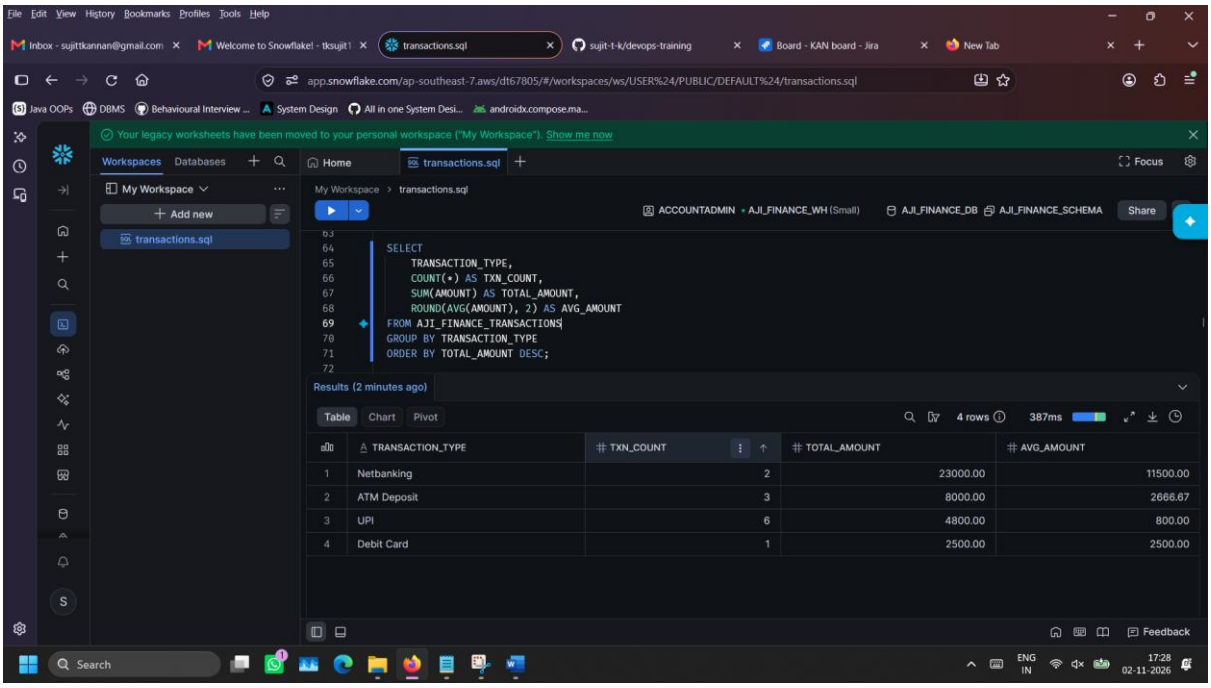
11.1) Project all records of the table (to verify whether all data has been loaded successfully)

```
SELECT COUNT(*) AS TOTAL_TRANSACTIONS
FROM AJI_FINANCE_DB.AJI_FINANCE_SCHEMA.AJI_FINANCE_TRANSACTIONS;
```



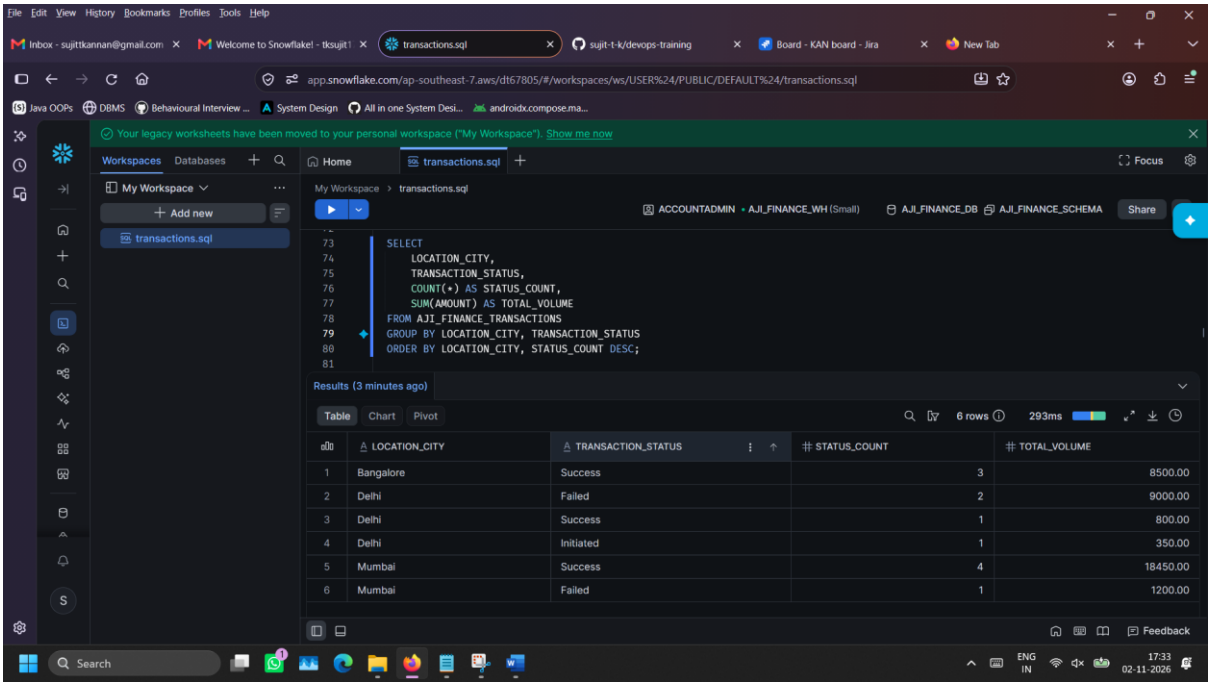
11.2) **Grouping & Aggregation** - **Retrieve transactions count, total amount and average amount of each transaction type**

```
SELECT
    TRANSACTION_TYPE,
    COUNT(*) AS TXN_COUNT,
    SUM(AMOUNT) AS TOTAL_AMOUNT,
    ROUND(AVG(AMOUNT), 2) AS AVG_AMOUNT
FROM AJI_FINANCE_TRANSACTIONS
GROUP BY TRANSACTION_TYPE
ORDER BY TOTAL_AMOUNT DESC;
```



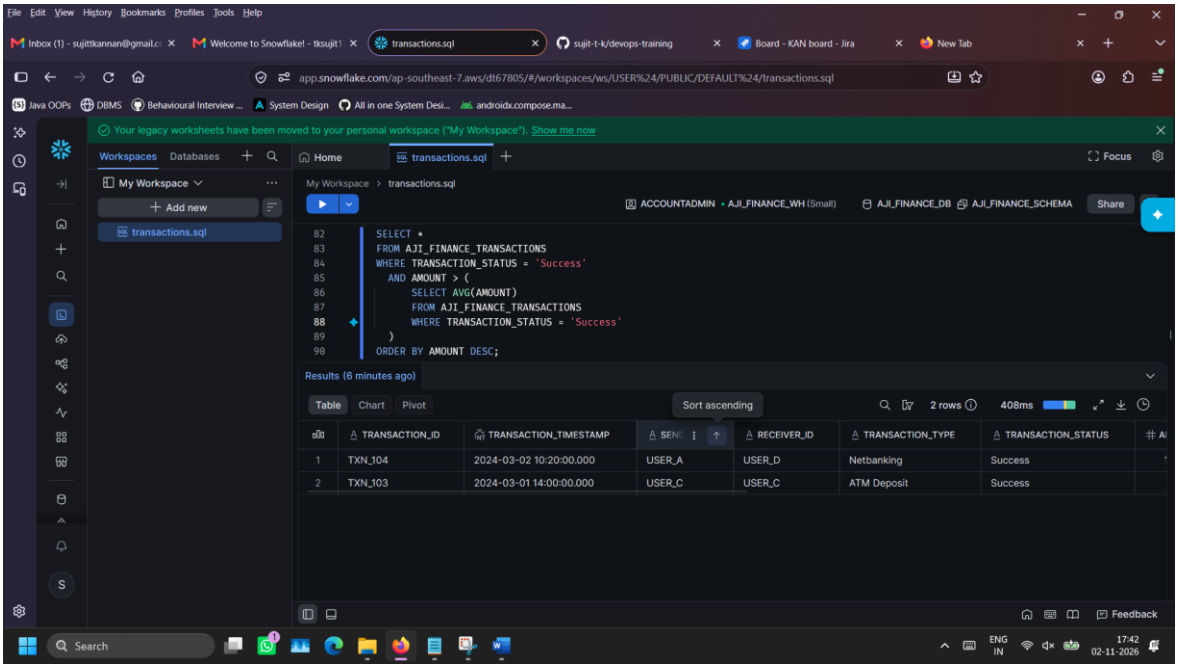
11.3) **Grouping, Aggregation & Sorting** - **Information of number of transactions initiated, failed and succeeded in each city**

```
SELECT
    LOCATION_CITY,
    TRANSACTION_STATUS,
    COUNT(*) AS STATUS_COUNT,
    SUM(AMOUNT) AS TOTAL_VOLUME
FROM AJI_FINANCE_TRANSACTIONS
GROUP BY LOCATION_CITY, TRANSACTION_STATUS
ORDER BY LOCATION_CITY, STATUS_COUNT DESC;
```



11.4) **Subquery execution – Identifying successful transaction which involves more than average transaction amount**

```
SELECT *
FROM AJI_FINANCE_TRANSACTIONS
WHERE TRANSACTION_STATUS = 'Success'
    AND AMOUNT > (
        SELECT AVG(AMOUNT)
        FROM AJI_FINANCE_TRANSACTIONS
        WHERE TRANSACTION_STATUS = 'Success'
    )
ORDER BY AMOUNT DESC;
```



12.)Deletion of database, warehouse and .sql file from workspace (so that additional costs doesn't incur):

```
DROP DATABASE IF EXISTS AJI_FINANCE_DB;
DROP WAREHOUSE IF EXISTS AJI_FINANCE_WH;
```

